01 / FEASIBILITY REPORT

Ride together. See each other.

Bendbound group rides and Sign in with Apple

Prepared 13 October 2026 | Repository review + primary-source research

Recommendation: introduce private, invite-only group rides for up to 10 riders, with native Sign in with Apple on iOS and explicit location-sharing consent. Reuse the existing GPS tracker and motorcycle icons. Build a separate real-time service, preferably on managed AWS infrastructure, so group traffic does not compete with route planning.

What the first release should do

A rider creates a group from a planned route or a free ride, shares a link or QR code, and starts when members are ready. Each connected rider appears on the map using the motorcycle selected in their settings. Everyone controls whether their own location is shared; the host controls membership and the room lifecycle.

Decision	Proposed first release
Identity	Sign in required only for group rides. Keep ordinary planning and local recordings usable without an account.
Location	Target one fresh update every 5 seconds while moving, subject to GPS, network and operating-system delivery.
Privacy	No public rider discovery, no permanent peer tracks, and a visible Stop sharing control.
Delivery	Estimated 4-6 developer-weeks including authentication, backend, app integration and device testing.

Reading guide

2 Existing foundations / 3 Rider experience / 4 Architecture / 5 Reliability
6 Apple login / 7 Privacy and security / 8 Cost / 9 Delivery and acceptance
10-11 Evidence and sources

This is an implementation proposal. App and server code were inspected locally; production resources were not audited or changed for this report. Timing, retention and performance targets below are proposed defaults, not measured guarantees.

02 / REPOSITORY FINDINGS

What Bendbound already has

Existing foundation	How it helps / what is missing
MapLibre ride map [C1]	RideMap renders the local rider and rotates its icon relative to map bearing. Add a remote-rider collection and one marker component per member.
Motorcycle settings [C2]	Ten existing artwork keys, including sports, adventure, cruiser and custom bikes. Send the selected key; each receiving device renders its bundled artwork.
Background GPS [C3]	expo-location + TaskManager record active rides. Reuse a single capture pipeline; add a separate sharing consumer and preserve local recording recovery.
Local persistence [C4]	Tours and settings live locally in SQLite storage. Login must preserve these records and must not imply automatic cloud synchronisation.
Route links [C5]	Universal links already open shared routes. Add /group/* to the association file and a group invite screen; current association covers /shared/* only.
AWS backend [C6]	Documented Caddy/EC2, Node catalogue and PostGIS setup. No group rooms, user identities or real-time sharing service were found in the inspected app/service paths.

Two boundaries need changing

The documented API uses a shared pilot bearer token that is extractable from the app bundle. It cannot identify a rider or authorise access to someone else's location. Routing is also limited to two concurrent requests, with per-IP and global request caps. A 10-person group can exceed these limits quickly. Group traffic needs its own authentication, endpoints and quotas. [C6]

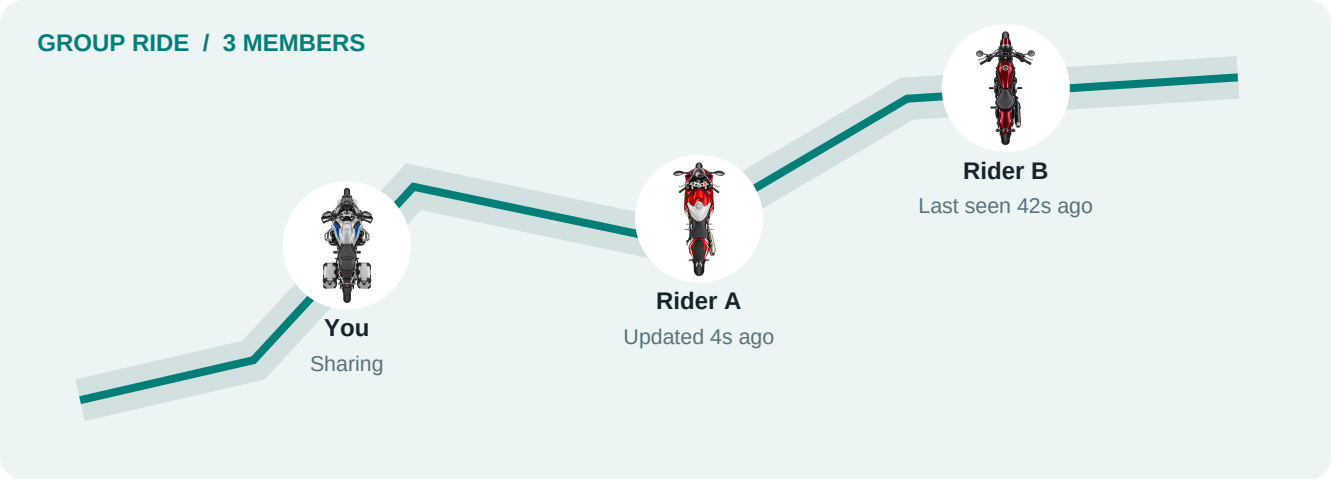
A useful but incomplete tracking foundation

The recorder asks for background permission, uses navigation-grade fixes, and requests deferred updates. It currently stops native GPS when a recording is paused. Group sharing must track its own active state: GPS remains required whenever recording *or* sharing is active. Update permission text and the Android notification to describe both purposes. [C3]

Scope control: add group state alongside the existing local ride state. Do not couple a network disconnect to stopping the recording, or make Apple login a prerequisite for offline route planning.

03 / PRODUCT PROPOSAL

The rider experience



Concept only: uses existing motorcycle artwork; not a screenshot of a working feature.

Step	Behaviour
Create	Choose an existing route or Free ride, set a group name, then generate an expiring invite. Suggested cap: 10 riders.
Join	Open an invite / QR code, sign in, choose a display name and review who will see your location. No sharing before explicit consent.
Ready and start	Lobby shows joined, sharing-ready, foreground-only and disconnected states. Host starts the room; late joins remain possible. Each rider starts their own navigation/recording.
Ride	Show all members with their selected bike, a distinct label/colour, GPS quality and update age. Offer Follow me and Fit group; keep local navigation as the default camera.
Leave or end	Each rider can stop sharing or leave at any time. Host can remove members and end the room. Ending the group stops sharing for all; local recording may continue.

Make connection and location status separate

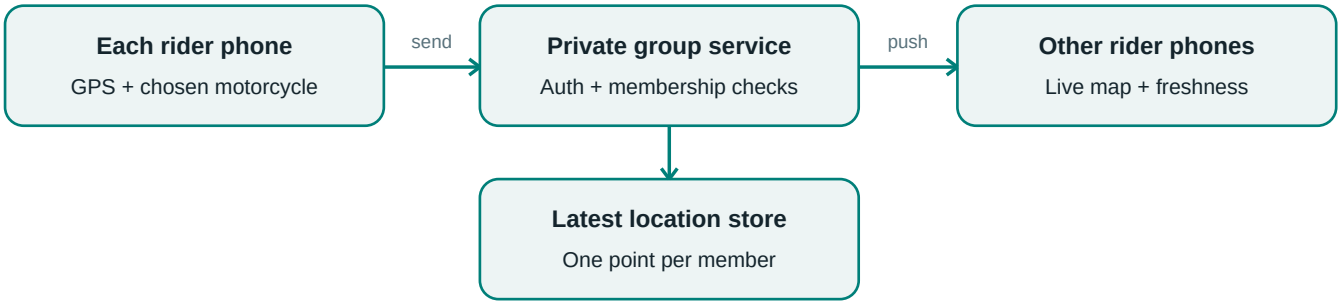
A socket can be connected while GPS is old. Use statuses such as Sharing, Waiting for GPS, Last seen 42s ago and Offline. Tapping a rider opens a compact card with name, bike and freshness. Avoid complex social interactions while riding.

Bike selection is part of the member profile

Seed motorcycleType from current settings on join and broadcast changes during a ride. Keep a server-side allowlist and a fallback for older app versions. Labels distinguish riders using the same bike. Remote position and heading use their own update stream, never the local rider's route projection. [C1, C2]

04 / ARCHITECTURE RECOMMENDATION

A separate service for live rides



Foreground: WebSocket | Background: HTTPS upload | All access restricted to room members

Recommended deployment

Use API Gateway WebSockets for foreground delivery, HTTP API/Lambda for room actions and background uploads, and DynamoDB for membership and latest fixes. A Lambda authorizer checks app sessions at connect; every publish/delivery checks membership and session validity. Disconnect revoked or expired sessions. This keeps group traffic separate from routing, in London.

Option	Assessment
Existing EC2 + separate Node service	Lowest initial infrastructure change. Own socket lifecycle, restarts, persistence, backups and scaling. Must measure memory/CPU headroom: the existing compose stack already allows substantial routing memory. Suitable for a small technical pilot. [C6]
Managed AWS real-time service	Recommended for a public MVP. Adds IAM, deployment configuration, connection registry and fan-out logic, but removes the need to keep an additional long-running socket host healthy.
Managed auth via Cognito	Optional identity alternative, independent of real-time hosting. Apple federation uses a Services ID and managed browser login; it is not a direct drop-in for native Expo Apple credentials. [R6]

Minimum data model

User: internal ID, provider subject, display name, motorcycleType. Room: owner, state, route reference/version, expiry. Member: role, sharing flag, device session and joined time. Latest fix: coordinates, accuracy, heading, speed, capturedAt, receivedAt, session epoch and sequence. Store connection IDs separately; one socket is not one identity.

Room API and events

Create/join/leave/end rooms, redeem invites, set sharing, remove members, upload fixes and fetch snapshots. Push member changes, location deltas and room-ended events. Reconnect through a fresh authorised snapshot. Make commands idempotent and reject duplicate/out-of-order fixes.

Keep route geometry in a versioned route snapshot. A public route-share link does not grant live location access. Reject group room reads when membership has ended, even if a previously issued app token is still valid.

05 / LOCATION AND BACKGROUND EXECUTION

Reliable enough to ride with

One GPS producer, two independent consumers

Refactor the existing task so recording and sharing can each keep GPS active. Local recording can ingest historical batches; live sharing sends only the newest sufficiently recent fix. Do not replay an offline GPS backlog as somebody's current position. On app restart, recover sharing conservatively and require fresh location before showing a rider as live. [C3]

Use the right transport for each app state

Foreground: maintain a WebSocket, upload throttled fixes and receive peer deltas. Background: attempt a bounded HTTPS upload from each location task invocation, with minimal work and no permanent retry queue. When foregrounded, reconnect and fetch a snapshot. This is a proposal requiring physical-device proof: background location execution does not promise uninterrupted network delivery. Apple documents suspension and queued updates; Expo documents termination and permission constraints. [R1, R2]

Proposed default	Reason / handling
5-second moving update target	Smooth remote markers between accepted fixes. Keep GPS capture separate from upload cadence. Test 2 seconds only if field latency requires it; use a longer stationary cadence.
Fresh <= 20 seconds	Show normal marker with real update age. Accuracy is a separate signal; a fresh, imprecise fix is not precise.
Stale > 20 seconds	Freeze animation and dim marker. Show last known age; never keep extrapolating a moving rider.
Offline >= 60 seconds	Show Offline / last seen. After 5 minutes hide the map point, retaining the member in the roster.
Room safety expiry	Suggested maximum 12 hours. End automatically unless deliberately extended; expiry must be enforced by the server, not only the phone.

Expected limitations

Screen lock must be tested with background location permission on real devices. Force-quitting, denied permissions, Android battery restrictions and no mobile coverage can interrupt sharing. Offer clearly labelled foreground-only participation. Expo Go is not the acceptance environment for background tracking. [R1]

Reconnect and freshness

Use exponential backoff with jitter, a per-device session epoch and monotonic sequence. Retain both original capture time and server receipt time; handle device clock skew explicitly. Server receipt time alone must never make an ancient GPS fix appear fresh. API Gateway connections last at most two hours and idle out after ten minutes, so planned reconnects and light heartbeats are required. [R7]

06 / IDENTITY DESIGN

Sign in with Apple

Preferred iOS experience: the native Apple sign-in sheet, backed by server verification and Bendbound app sessions. Keep login optional for ordinary app use, and require it when creating or joining a group. Apple proves identity; room membership determines who can see location.

App work

Add SDK-compatible expo-apple-authentication and expo-secure-store; enable the Apple capability and ios.usesAppleSignIn. Use the official Apple button. Handle cancellation without an error screen. The Expo native module supports iOS, not Android/web; capability changes need a new signed binary. Save first-sign-in name if supplied, but allow a rider nickname and missing email/name. [R3]

Backend work

Issue a one-time sign-in challenge. Send the native authorization code and identity token to the server; verify Apple signature against trusted keys, issuer, audience, expiry and nonce. Exchange the code server-side and associate the verified subject with an internal user ID. Issue short-lived Bendbound access tokens and rotating, revocable refresh sessions. Never trust a client-supplied user ID or decode a JWT without verification. [R4]

Secret and session storage

Keep the Apple signing key and server token-signing material in AWS Secrets Manager with least-privilege access. No private key or app client secret belongs in EXPO_PUBLIC variables. Store app refresh credentials in SecureStore rather than current SQLite settings; background access must work while the screen is locked, so explicitly test the chosen keychain accessibility. [R5]

Cognito alternative and Android path

Cognito can manage accounts and application sessions through its Apple social provider using a Services ID, registered return URL and Apple key. Its normal federation flow is browser OAuth; use a system authentication session with authorization-code + PKCE, not an embedded WebView. A raw native Apple token is not automatically a Cognito user-pool session. Choose this alternative if managed identity is preferred over native-sheet UX and custom session ownership. [R6, R13]

If Android is in the first launch, implement Apple web OAuth with a Services ID or add another supported provider. Keep a common internal identity model and link accounts only through verified ownership; never merge by email alone. Browser and native Apple identifiers must be configured consistently. This is additional scope, not covered by adding the iOS module.

Deletion and revocation are part of launch

Provide in-app account deletion; remove memberships, stop sharing, revoke Bendbound sessions and revoke Apple tokens. Support Apple credential-revoked events and server-side lifecycle handling. Cognito user deletion alone is not a complete Apple-token revocation design. Decide explicitly whether local, device-owned recordings are also deleted. [R8, R9]

07 / SECURITY AND DATA HANDLING

Private by default

Control	Proposed implementation
Invite protection	High-entropy expiring links, QR codes and optional host approval. Rate-limit short join codes and rotate/revoke invitations. Hide tokens from analytics, logs and link previews.
Room access	Authenticate every room action. Check membership on reads, sends and fan-out. Server assigns rider ID; users can publish only their own fix. Removing a rider closes their room access immediately.
Sharing consent	Each rider enables sharing per session. Stop sharing clears their latest point and tells peers to remove it. Logout and deletion terminate sharing before clearing credentials.
Data minimisation	Persist latest position only, with an access expiry. Do not collect other riders' full tracks. Use a purge job; DynamoDB TTL is asynchronous and can leave expired items for days. [R10]
Suggested retention	Clear points on stop/leave/end; purge any missed coordinates within 15 minutes. Room metadata expires within 24 hours after ending. Keep account profile until deletion; redact location from backups and logs.
Abuse and resilience	Validate bounds, payload size, sequence and clock skew; cap update rates. Limit group size and rooms per account. Do not claim to prevent all GPS spoofing.
Operations	TLS, least-privilege IAM, secret rotation, short log retention and alarms for stale uploads, fan-out failure and auth errors. Counts and timing metrics should avoid rider coordinates.

UK privacy work before wider release

Treat identifiable rider locations as personal data. Explain who sees them, when sharing ends and how deletion works. Document the lawful basis and complete a DPIA screening; location monitoring and scale are relevant risk factors in ICO guidance. Opting into a product switch is not a substitute for the broader data-protection assessment. [R11]

Device and recipient boundaries

Peers can retain screenshots or externally record what they see; deletion cannot recall their copies. Make that clear in the sharing notice. Allow members to leave without the host's permission. When the host disappears, the room continues until explicit end or expiry; do not automatically end everyone's session on a temporary signal loss.

Release condition: prove that an invited-but-not-joined user, a removed member and a user holding an old token cannot retrieve or receive current room locations.

08 / LONDON-REGION PLANNING ESTIMATE

What it could cost

Infrastructure cost is likely modest at pilot volume. The main investment is engineering and real-device testing. Figures below are USD, exclude tax, and do not include existing routing costs. They are estimates rather than a quote.

Verified WebSocket rates

AWS's public eu-west-2 price list currently lists $1.19 per million WebSocket messages for the first billion messages/month and $0.297 per million connection minutes. API Gateway meters messages in 32 KB increments. Small rider deltas fit within one unit. [R12, R14]

Example: 10 riders, 2-hour session, one update per rider every 5 seconds; all ten phones stay connected. Each update is delivered to the other nine members. This upper-bound foreground scenario counts inbound WebSocket messages plus recipient deliveries; background HTTP uploads have separate HTTP/Lambda costs.

Monthly volume	Messages / minutes	WebSocket subtotal
1 group session	144,000 messages 1,200 connection minutes	$0.17
100 group sessions	14.4 million messages 120,000 connection minutes	$17.17
1,000 group sessions	144 million messages 1.2 million connection minutes	$171.72

Calculation per session: 7,200 / 5 = 1,440 updates/rider; 10 x 1,440 x (1 inbound + 9 outbound) = 144,000 messages. Cost = messages/1m x 1.19 + minutes/1m x 0.297. Excludes snapshots, joins, acknowledgements, heartbeats and retries. Fan-out grows roughly with the square of group size.

Other charges and a working budget

Add HTTP API calls, Lambda invocations/runtime, DynamoDB reads/writes, data transfer, logs and secrets. Fan-out implementation matters: one Lambda call per recipient is more expensive than one call per accepted update. For 100 sessions/month, reserve **$30-$75/month incremental infrastructure** as a provisional planning allowance, then validate with measured traffic and an AWS calculator. This range is an engineering allowance, not a priced bill of materials.

Authentication choice

Native Apple + custom Bendbound sessions adds backend/session operations rather than a per-user Cognito fee. If choosing Cognito Lite or Essentials, published pricing includes 10,000 direct/social MAUs free per account or organisation; Apple social federation is distinct from generic enterprise OIDC pricing. Existing account usage affects the allowance. [R15]

Existing EC2 alternative

EC2 may have low marginal cost if spare capacity exists. Capacity is untested; budget for operations and potential resizing.

09 / MILESTONES AND LAUNCH GATES

A practical delivery plan

Phase	Deliverable	Effort estimate
1. Prove background sharing	Two physical iPhones: screen-locked HTTPS publishing, foreground peer map, poor-network recovery. Prototype the selected auth path and native capability.	3-5 days
2. Identity and rooms	Apple verification + sessions, deletion/revocation, room lifecycle, invites, membership and authorised WebSocket snapshots.	5-7 days
3. App integration	Lobby, selected bike markers, independent sharing/recording state, freshness UI, consent and controls, route snapshot.	5-7 days
4. Harden and pilot	Multi-device field testing, permissions, privacy text, metrics, recovery, load/security checks and signed TestFlight build.	5-8 days

Total: approximately 18-27 focused developer days, or 4-6 full-time weeks. Assumes one experienced developer, iOS-first release, existing Apple account access and no chat, public discovery or shared route editing. Android parity or a production native-to-managed identity bridge adds work. Estimates are based on code review, not a completed prototype.

Acceptance matrix

Use 2, 5 and 10 physical riders/devices. A simulator helps test map behaviour but cannot validate screen-lock reliability.

Scenario	Pass criterion
Moving / rotated map	Each rider shows their selected bike and correct heading; duplicate bike choices remain distinguishable. Aim for p95 remote age under 10s on good coverage.
Screen lock / battery	30-60 minute locked-screen ride publishes when permitted; measure power against the existing recorder. No promise of a fixed five-second OS schedule.
Signal loss / reconnect	Stale state appears at the threshold; no animated phantom movement. Newest fresh fix wins after reconnection; historical batches cannot overwrite it.
Stop / pause / expiry	Stopping sharing removes location promptly online; offline publication ceases locally and server expiry applies. Pausing recording does not silently stop agreed sharing.
Access / lifecycle	Wrong-room and removed-member access fails. End, logout, account deletion and invite revocation are enforced; old sessions cannot bypass checks.
Route independence	Group connection loss leaves local navigation and recording usable. Room route reference remains stable if a public share expires.

Suggested launch gate: complete Phase 1 before committing to the full implementation. Publish limits honestly, use a capped private TestFlight cohort first, and monitor measured freshness and battery use before wider rollout.

10 / AUDIT TRAIL

Evidence from this repository

Inspected on 13 October 2026. References describe the local working tree, which includes ongoing uncommitted work. They are not a claim that the same revision is currently deployed. No credential values were copied into this report.

C1 src/components/RideMap.tsx; src/hooks/useRiderMotion.ts

Local motorcycle marker, map-bearing rotation, map motion and route layers.

C2 src/components/MotorcycleGlyph.tsx; src/data/motorcycles.ts; src/types.ts

Bundled motorcycle artwork, allowed keys and settings.motorcycleType.

C3 src/services/rideTracking.ts; src/services/rideStore.ts; app.json

Top-level background task, permission handling, start/pause/end/recovery and native background configuration.

C4 src/state/AppProvider.tsx; package.json

SQLite settings/tours persistence; Expo SDK 57 dependencies. No auth/session dependencies in the inspected package manifest.

C5 routing/catalogue/service/route-shares.mjs; app.json

Public route shares, bendbound scheme, associated domain and /shared/* association paths.

C6 docs/routing-ec2.md; routing/catalogue/compose.yml

Documented EC2/Caddy backend, extractable pilot bearer token, request limits and container memory limits.

Research method

Compared the existing map, settings, GPS capture, persistence and server boundaries with current Expo SDK 57, Apple, AWS and ICO documentation. Proposed choices and effort estimates are engineering judgments. Cost arithmetic uses the public AWS London API Gateway price list; no secret or authenticated AWS billing access was required.

Open decisions before implementation

Confirm iOS-only versus simultaneous Android launch; native Apple UX versus Cognito-managed identity; whether guests should be allowed in a later release; maximum group size; whether route changes should ever synchronise mid-ride; and the final privacy/retention policy. The recommended MVP assumes iOS-first, authenticated participants and a fixed shared route snapshot.

11 / REFERENCE LINKS

Primary sources

All links checked 13 October 2026. Click a source title to open the documentation. Prices and platform behaviour should be rechecked when implementation begins.

R1 Expo SDK 57: Location
Background permissions, native-build testing and termination limits.

R2 Apple: Handling location updates in the background
App suspension, queued fixes and background capability.

R3 Expo SDK 57: AppleAuthentication
iOS support, native capability and first-sign-in profile fields.

R4 Apple: Verifying a user
Signature, issuer, audience, expiry and nonce checks.

R5 Expo SDK 57: SecureStore
Protected credential storage and keychain accessibility.

R6 AWS: Apple as a Cognito social identity provider
Services ID, key and managed-login configuration.

R7 AWS: API Gateway WebSocket quotas
Two-hour connection limit and ten-minute idle timeout.

R8 Apple: Offering account deletion in your app
In-app deletion and Apple token revocation expectations.

R9 Apple TN3194: Account deletion and token revocation
Server-side revocation and credential lifecycle.

R10 AWS: DynamoDB time to live
Expired records can remain for days; TTL is not immediate deletion.

R11 ICO: When do we need to do a DPIA?
Risk screening, monitoring and scale.

R12 AWS: API Gateway pricing
Message units, connection billing and additional service charges.

R13 AWS: Cognito authorization endpoint
Authorization-code flow and PKCE.

R14 AWS Price List: API Gateway, EU London
eu-west-2 WebSocket messages $1.19/million; minutes $0.297/million.

R15 AWS: Cognito pricing
Direct/social MAU allowance and pricing tiers.